

Automatisation de la génération et de l'affichage de noeuds premiers alternés

Nathan Landau Marc Descamps

Numéro de candidat : 23045

Plan

▶ 1. Séquences DT

- ▶ Noeud, premier, alterné...
- ▶ Séquence DT
- ▶ Énumération des séquences DT standard
- ▶ Filtrage des séquences

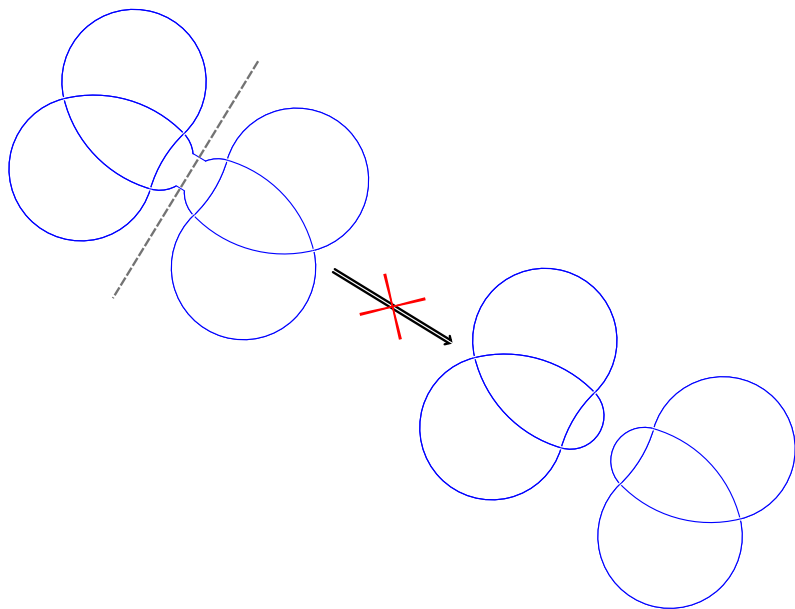
▶ 2. Transformations de graphes

- ▶ Dessin de Lombardi
- ▶ Graphe de Tait, graphe médial, graphe des angles réduit
- ▶ Empilement des cercles primaux/duaux

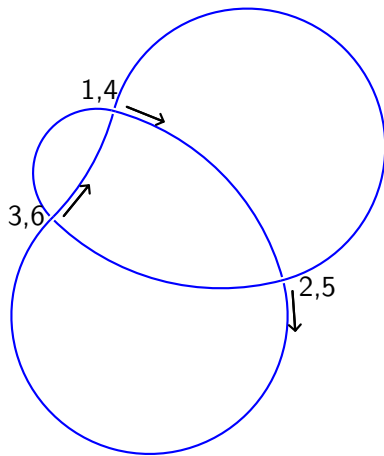
▶ 3. Calculs géométriques et résultats

- ▶ Calcul des rayons (descente de gradient)
- ▶ Calcul des positions
- ▶ Calculs géométriques
- ▶ Résultats

Noeud, premier, alterné...



Séquence DT



$$(1, 4), (3, 6), (5, 2) \implies 2 \ 6 \ 4$$

Énumération des séquences DT standard

Filtrage des séquences

Dessin de Lombardi

Graphe de Tait, graphe médial, graphe des angles réduit

Empilement des cercles primaux/duaux

Calcul des rayons (descente de gradient)

Calcul des positions

Calculs géométriques

Résultats

Annexe (I) (1/26) I

```
1 struct seq_dt {
2     int *seq; // on stocke seulement les nombre pairs, divisés
3     par 2
4     int taille; //nombre de croisements
5 };
6 typedef struct seq_dt seq_dt;
7
8 void init_seq_dt(seq_dt *s, int taille) {
9     s->seq = (int *)malloc(taille * sizeof(int));
10    if (s->seq == NULL) {
11        fprintf(stderr, "Pas pu allouer de la mémoire\n");
12        exit(EXIT_FAILURE);
13    }
14    s->taille = taille;
15 }
16
17 void free_seq_dt(seq_dt *s) {
18     if (s->seq != NULL) {
19         free(s->seq);
20         s->seq = NULL;
```

Annexe (I) (2/26) I

```
1     }
2     s->taille = 0;
3 }
4
5 void print_seq_dt(seq_dt *s) {
6     if (s->seq == NULL || s->taille <= 0) {
7         printf("La séquence est vide.\n");
8         return;
9     }
10    for (int i = 0; i < s->taille; i++) {
11        printf("(%d; %d), ", 2*i+1, 2*s->seq[i]);
12    }
13    printf("\n");
14 }
15
16 void fprintf_seq_dt(seq_dt *s, FILE* f){
17     fprintf(f, "%d ", s->taille);
18     for(int i = 0; i<s->taille; i++){
19         fprintf(f, "%d ", s->seq[i]);
20     }
```

Annexe (I) (3/26) I

```
1    fprintf(f, "\n");
2 }
3
4 int abs(int x) {
5     return (x < 0) ? -x : x;
6 }
7
8 void calculer_a_i(int* p, seq_dt* s){    //On remplit les cases
    de 1 à 2n par leur binôme dans 1 à 2n
9     for(int i=0; i<s->taille; i++){
10         p[2*i+1] = 2*s->seq[i];
11         p[2*s->seq[i]] = 2*i+1;
12     }
13 }
14
15 void calculer_phi(int** phi, int* a, int n){
16     for(int i=1; i<=2*n; i++){
17         phi[i][i] = 1;
18         for(int r=i+1; r<=2*n; r++){
19             if(a[r]<= a[i] && i <= a[r]){
20                 phi[i][r] = -phi[i][r-1];
```

Annexe (I) (4/26) I

```
1         } else {
2             phi[i][r] = phi[i][r-1];
3         }
4     }
5     for(int r=i-1; r>0; r--){
6         if(a[r+1]<= a[i] && i <= a[r+1]){
7             phi[i][r] = -phi[i][r+1];
8         } else {
9             phi[i][r] = phi[i][r+1];
10        }
11    }
12 }
13 }
14
15 struct bst{
16     seq_dt val;
17     struct bst *g, *d;
18 };
19
20 typedef struct bst bst;
```

Annexe (I) (5/26) I

```
1
2 int comp(seq_dt a, seq_dt b){
3     for (int i = 0; i < a.taille; i++){
4         if (a.seq[i] < b.seq[i]){
5             return 1;
6         } else if (a.seq[i] > b.seq[i]) {
7             return -1;
8         }
9     }
10    return 0;
11 }
12
13 seq_dt copy_dt(seq_dt s){
14     seq_dt s2;
15     s2.taille = s.taille;
16     s2.seq = malloc(sizeof(int) * s2.taille);
17     for (int i = 0; i < s2.taille; i++){
18         s2.seq[i] = s.seq[i];
19     }
20     return s2;
```

Annexe (I) (6/26) I

```
1  }
2
3  void inserer_bst(bst** arbre, seq_dt s){
4      if (*arbre == NULL) {
5          *arbre = malloc(sizeof(bst));
6          (*arbre)->val = copy_dt(s);
7          (*arbre)->g = NULL;
8          (*arbre)->d = NULL;
9      }
10     int c = comp((*arbre)->val, s);
11     if (c == -1){
12         inserer_bst(&((*arbre)->g), s);
13     } else if (c == 1){
14         inserer_bst(&((*arbre)->d), s);
15     }
16 }
17
18 bool trouver_bst(bst** arbre, seq_dt s){
19     if (*arbre == NULL) {
20         return false;
```

Annexe (I) (7/26) I

```
1     }
2     int c = comp((*arbre)->val, s);
3     if (c == -1){
4         return trouver_bst(&((*arbre)->g), s);
5     } else if (c == 1){
6         return trouver_bst(&((*arbre)->d), s);
7     }
8     return true;
9 }
10
11
12 // ordre : ordre des permutations, puis ordre des sous parties
pour les signes
13 void next_seq_dt(seq_dt *s) {
14     if (s->seq == NULL || s->taille <= 0) {
15         fprintf(stderr, "La séquence n'est pas initialisée ou a  
une taille invalide.\n");
16         return;
17     }
18
19     int pivot_index = s->taille - 2;
20     while (pivot_index >= 0 && s->seq[pivot_index + 1] <= s->seq  
[pivot_index]) {
```

Annexe (I) (7/26) II

Annexe (I) (8/26) I

```
1      pivot_index--;
2  }
3
4  if (pivot_index < 0) {
5      printf("Fin de la séquence.\n");
6      free_seq_dt(s);
7      exit(EXIT_SUCCESS);
8      return;
9  }
10
11  int swap_index = s->taille - 1;
12  while (s->seq[swap_index] <= s->seq[pivot_index]) {
13      swap_index--;
14  }
15
16  int temp = s->seq[pivot_index];
17  s->seq[pivot_index] = s->seq[swap_index];
18  s->seq[swap_index] = temp;
19
20  for (int i = pivot_index + 1, j = s->taille - 1; i < j; i++,  
      j--) {
```

Annexe (I) (9/26) I

```
1      int temp = s->seq[i];
2      s->seq[i] = s->seq[j];
3      s->seq[j] = temp;
4  }
5  }
6
7  typedef seq_dt r_seq_dt;
8
9  r_seq_dt* realiser_seq_dt(seq_dt *s){
10     r_seq_dt *res = malloc(sizeof(r_seq_dt));
11     init_seq_dt(res, 2*s->taille);
12     if (s->seq == NULL || s->taille <= 0) {
13         return res;
14     }
15     for (int i = 0; i < s->taille; i++) {
16         res->seq[2*i] = 2*i+1;
17         res->seq[2*i+1] = 2*s->seq[i];
18     }
19     return res;
20 }
```

Annexe (I) (10/26) I

```
1
2 void next_seq_dt_std(seq_dt *s, bst** arbre) {
3     // print_seq_dt(s);
4     while(trouver_bst(arbre, *s)){
5         // print_seq_dt(s);
6         next_seq_dt(s);
7     }
8     // printf("nouvelle\n");
9     //on calcule les seq_dt équivalentes
10    seq_dt next = copy_dt(*s);
11    inserer_bst(arbre, next);
12    //Celles dans le même sens
13    for(int decal = 0; decal < s->taille; decal++){
14        int last = next.seq[0];
15        for(int i = 0; i<next.taille-1; i++){
16            next.seq[i] = next.seq[i+1] - 1;
17            if(next.seq[i] == 0) next.seq[i] = s->taille;
18        }
19        next.seq[next.taille-1] = last - 1;
20        if(next.seq[next.taille-1] == 0) next.seq[next.taille-1]
            = s->taille;
```

Annexe (I) (11/26) I

```
1      insérer_bst(arbre, next);
2  }
3  r_seq_dt* r = realiser_seq_dt(&next);
4  //Decalage de 1
5  for(int i = 0; i<r->taille; i++){
6      r->seq[i] = r->seq[i]-1;
7      if(r->seq[i] == 0) r->seq[i] = r->taille;
8  }
9  for(int i = 0; i< s->taille; i++){
10     assert((r->seq[2*i+1]-1)%2 == 0);
11     assert((r->seq[2*i])%2 == 0);
12     next.seq[(r->seq[2*i+1]-1)/2] = r->seq[2*i]/2;
13 }
14 //De 2 en 2
15 for(int decal = 0; decal < s->taille; decal++){
16     int last = next.seq[0];
17     for(int i = 0; i<next.taille-1; i++){
18         next.seq[i] = next.seq[i+1] - 1;
19         if(next.seq[i] == 0) next.seq[i] = s->taille;
20     }
```

Annexe (I) (12/26) I

```
1      next.seq[next.taille-1] = last - 1;
2      if(next.seq[next.taille-1] == 0) next.seq[next.taille-1]
   = s->taille;
3      inserer_bst(arbre, next);
4  }
5
6  //Changement de sens
7  free_seq_dt(&next);
8  next = copy_dt(*s);
9  r = realiser_seq_dt(&next);
10 for(int i = 0; i<r->taille; i++){
11     r->seq[i] = r->taille + 1 - r->seq[i];
12     r->seq[i] = r->seq[i] - 1;
13     if(r->seq[i] == 0) r->seq[i] = r->taille;
14 }
15 for(int i = 0; i< s->taille; i++){
16     assert((r->seq[2*i+1])%2 == 0);
17     assert((r->seq[2*i]-1)%2 == 0);
18     next.seq[(r->seq[2*i]-1)/2] = r->seq[2*i+1]/2;
19 }
20 inserer_bst(arbre, next);
```

Annexe (I) (13/26) I

```
1 //Celles dans l'autre sens
2 for(int decal = 0; decal < s->taille; decal++){
3     int last = next.seq[0];
4     for(int i = 0; i<next.taille-1; i++){
5         next.seq[i] = next.seq[i+1]- 1;
6         if(next.seq[i] == 0) next.seq[i] = s->taille;
7     }
8     next.seq[next.taille-1] = last - 1;
9     if(next.seq[next.taille-1] == 0) next.seq[next.taille-1]
10    = s->taille;
11    inserer_bst(arbre, next);
12 }
13 r = realiser_seq_dt(&next);
14 //Decalage de 1
15 for(int i = 0; i<r->taille; i++){
16     r->seq[i] = r->seq[i]-1;
17     if(r->seq[i] == 0) r->seq[i] = r->taille;
18 }
19 for(int i = 0; i< s->taille; i++){
20     assert((r->seq[2*i+1]-1)%2 == 0);
21     assert((r->seq[2*i])%2 == 0);
```

Annexe (I) (14/26) I

```
1      next.seq[(r->seq[2*i+1]-1)/2] = r->seq[2*i]/2;
2  }
3  inserer_bst(arbre, next);
4  //De 2 en 2
5  for(int decal = 0; decal < s->taille; decal++){
6      int last = next.seq[0];
7      for(int i = 0; i<next.taille-1; i++){
8          next.seq[i] = next.seq[i+1] - 1;
9          if(next.seq[i] == 0) next.seq[i] = s->taille;
10     }
11     next.seq[next.taille-1] = last - 1;
12     if(next.seq[next.taille-1] == 0) next.seq[next.taille-1]
13     = s->taille;
14     inserer_bst(arbre, next);
15 }
16
17 free_seq_dt(&next);
18
19 void next_seq_dt_std2(seq_dt *s, bst** arbre) {
20     // print_seq_dt(s);
```

Annexe (I) (15/26) I

```
1  while(trouver_bst(arbre, *s)){
2      // print_seq_dt(s);
3      next_seq_dt(s);
4  }
5  // printf("nouvelle\n");
6  //on calcule les seq_dt équivalentes
7  seq_dt next = copy_dt(*s);
8  inserer_bst(arbre, next);
9
10 r_seq_dt* r = realiser_seq_dt(&next);
11 for(int decal = 0; decal < r->taille; decal++){
12     //Decalage de 1
13     for(int i = 0; i < r->taille; i++){
14         r->seq[i] = r->seq[i]-1;
15         if(r->seq[i] == 0) r->seq[i] = r->taille;
16     }
17     if(r->seq[0] % 2 == 0){
18         for(int i = 0; i < s->taille; i++){
19             assert((r->seq[2*i+1]-1)%2 == 0);
20             assert((r->seq[2*i])%2 == 0);
```

Annexe (I) (16/26) I

```
1         next.seq[(r->seq[2*i+1]-1)/2] = r->seq[2*i]/2;
2     }
3     } else {
4         for(int i = 0; i< s->taille; i++){
5             assert((r->seq[2*i+1])%2 == 0);
6             assert((r->seq[2*i]-1)%2 == 0);
7             next.seq[(r->seq[2*i]-1)/2] = r->seq[2*i+1]/2;
8         }
9     }
10    inserer_bst(arbre, next);
11}
12
13
14//Changement de sens
15free_seq_dt(&next);
16next = copy_dt(*s);
17r = realiser_seq_dt(&next);
18for(int i = 0; i<r->taille; i++){
19    r->seq[i] = r->taille + 1 - r->seq[i];
20    r->seq[i] = r->seq[i] - 1;
```

Annexe (I) (17/26) I

```
1      if(r->seq[i] == 0) r->seq[i] = r->taille;
2  }
3  for(int i = 0; i < s->taille; i++){
4      assert((r->seq[2*i+1])%2 == 0);
5      assert((r->seq[2*i]-1)%2 == 0);
6      next.seq[(r->seq[2*i]-1)/2] = r->seq[2*i+1]/2;
7  }
8  inserer_bst(arbre, next);
9  //Celles dans l'autre sens
10 r = realiser_seq_dt(&next);
11 for(int decal = 0; decal < r->taille; decal++){
12     //Decalage de 1
13     for(int i = 0; i < r->taille; i++){
14         r->seq[i] = r->seq[i]-1;
15         if(r->seq[i] == 0) r->seq[i] = r->taille;
16     }
17     if(r->seq[0] % 2 == 0){
18         for(int i = 0; i < s->taille; i++){
19             assert((r->seq[2*i+1]-1)%2 == 0);
20             assert((r->seq[2*i])%2 == 0);
```

Annexe (I) (18/26) I

```
1         next.seq[(r->seq[2*i+1]-1)/2] = r->seq[2*i]/2;
2     }
3 } else {
4     for(int i = 0; i < s->taille; i++){
5         assert((r->seq[2*i+1])%2 == 0);
6         assert((r->seq[2*i]-1)%2 == 0);
7         next.seq[(r->seq[2*i]-1)/2] = r->seq[2*i+1]/2;
8     }
9 }
10 inserer_bst(arbre, next);
11 }
12
13 free_seq_dt(&next);
14 }
15
16 bool regle1(seq_dt* s);
17
18 void print_bst(bst* arbre){
19     if (arbre == NULL) {
20         return;
```

Annexe (I) (19/26) I

```
1      }
2      print_bst(arbre->g);
3      if(regle1(&(arbre->val))){
4          print_seq_dt(&(arbre->val));
5      }
6      print_seq_dt(&(arbre->val));
7      print_bst(arbre->d);
8  }
9
10
11 int * init_UF(int n){
12     int *arbre = malloc(sizeof(int) * n);
13     for (int i = 0; i < n; i++){
14         arbre[i] = i;
15     }
16     return arbre;
17 }
18
19 int Find(int arbre[], int A){
20     if (arbre[A] == A){
```

Annexe (I) (20/26) I

```
1      return A;
2  }
3  arbre[A] = Find(arbre, arbre[A]);
4  return arbre[A];
5  }
6
7  void Union(int arbre[], int A, int B){
8      arbre[Find(arbre, A)] = Find(arbre, B);
9  }
10
11 int min(int a, int b){
12     if (a < b){
13         return a;
14     }
15     return b;
16 }
17
18 int max(int a, int b){
19     if (a > b){
20         return a;
```

Annexe (I) (21/26) I

```
1      }
2      return b;
3  }
4
5  bool regle1(seq_dt *seq){
6      if (seq->taille < 3){
7          return false;
8      }
9      int *arbre = init_UF(seq->taille);
10     for (int i = 0; i < seq->taille; i++){
11         for (int j = 0; j < i; j++){
12             if (Find(arbre, i) == Find(arbre, j)){
13                 continue;
14             }
15             int d1 = min(abs(seq->seq[i]) * 2, i * 2 + 1),
16                 d2 = min(abs(seq->seq[j]) * 2, j * 2 + 1),
17                 f1 = max(abs(seq->seq[i]) * 2, i * 2 + 1),
18                 f2 = max(abs(seq->seq[j]) * 2, j * 2 + 1);
19
20             if ((d1 < d2 && d2 < f1 && f1 < f2) || (d2 < d1 &&
d1 < f2 && f2 < f1)){
```

Annexe (I) (22/26) I

```
1         Union(arbre, i, j);
2     }
3 }
4 }
5
6     for (int i = 1; i < seq->taille; i++){
7         if (Find(arbre, 0) != Find(arbre, i)){
8             free(arbre);
9             return false;
10        }
11    }
12    free(arbre);
13    return true;
14 }
15
16 void test_regle1(){
17     int taille = 7;
18
19     seq_dt s;
20     init_seq_dt(&s, taille);
```

Annexe (I) (23/26) I

```
1
2     s.seq[0] = 6;
3     s.seq[1] = 4;
4     s.seq[2] = 5;
5     s.seq[3] = 2;
6     s.seq[4] = 3;
7     s.seq[5] = 7;
8     s.seq[6] = 1;
9
10    printf("%s\n", regle1(&s)==false?"correct":"faux");
11    free_seq_dt(&s);
12
13    taille = 3;
14    init_seq_dt(&s, taille);
15
16    s.seq[0] = 2;
17    s.seq[1] = 3;
18    s.seq[2] = 1;
19
20    printf("%s\n", regle1(&s)==true?"correct":"faux");
```

Annexe (I) (24/26) I

```
1    free_seq_dt(&s);
2
3    taille = 5;
4    init_seq_dt(&s, taille);
5
6    s.seq[0] = 3;
7    s.seq[1] = 4;
8    s.seq[2] = 5;
9    s.seq[3] = 1;
10   s.seq[4] = 2;
11
12   printf("%s\n", regle1(&s)==true?"correct":"faux");
13   free_seq_dt(&s);
14
15   taille = 6;
16   init_seq_dt(&s, taille);
17
18   s.seq[0] = 5;
19   s.seq[1] = 3;
20   s.seq[2] = 4;
```

Annexe (I) (25/26) I

```
1      s.seq[3] = 2;
2      s.seq[4] = 6;
3      s.seq[5] = 1;
4
5      printf("%s\n", regle1(&s)==false?"correct":"faux");
6      free_seq_dt(&s);
7  }
8
9  int main(){
10     int taille = 7;
11     seq_dt s;
12     init_seq_dt(&s, taille);
13
14     for (int i = 1; i <= taille; i++) {
15         s.seq[i-1] = i;
16     }
17     // int seq[7] = {2, 4, 6, 7, 1, 3, 5};
18     // s.seq = seq;
19     FILE* f = fopen("sequences.txt", "w");
20     bst* arbre = NULL;
```

Annexe (I) (26/26) I

```
1  while(true){
2      if(regle1(&s)){
3          fprintf_seq_dt(&s, f);
4      }
5      next_seq_dt_std2(&s, &arbre);
6  }
7  fclose(f);
8  // inserer_bst(&arbre, s);
9  // print_bst(arbre);
10
11
12  return 0;
13 }
```

Annexe (II) (1/35) I

```
1 struct voisin {
2     int id_arete;
3     int sommet;
4 };
5
6 struct plongement {
7     int n;
8     voisin **adj; //dans la case i : les voisins du sommet i,
9                  //dans leur ordre combinatoire
10    int *deg; //dans la case i : les voisins du sommet i, dans
11              //leur ordre combinatoire
12 };
13
14 struct face {
15     int n_sommets;
16     voisin *sommets;
17 };
18
19 struct faces_arr {
20     int n;
21     face *faces;
22 };
```

Annexe (II) (2/35) I

```
1
2 struct graphe_tait {
3     int n;
4     face *sommets;
5     int **adj;
6     int *deg;
7 };
8
9 struct graphe {
10     int n;
11     int_vector **adj;
12 };
13
14 struct graphe_angles_reduit {
15     graphe H;
16     int *cycle_exterieur;
17 };
18
19 struct arete {
20     int x, y;
```

Annexe (II) (3/35) I

```
1 };
2
3 struct arete_arr {
4     int n;
5     arete *aretes;
6 };
7
8 struct vec {
9     double x, y;
10 };
11
12 faces_arr calculer_faces(plongement p){
13     int n = p.n;
14     bool **vus = malloc(sizeof(bool*) * n);
15     for (int i = 0; i < n; i++){
16         vus[i] = malloc(sizeof(bool) * p.deg[i]);
17         for (int j = 0; j < p.deg[i]; j++){
18             vus[i][j] = false;
19         }
20     }
```

Annexe (II) (4/35) I

```
1      face *f = malloc(sizeof(face) * n * 4); // le nombre de
2      faces est au plus de  $4n$  car le graphe est 4-régulier
3      int i_face = 0;
4      voisin *proc_sommet = malloc(sizeof(voisin) * n);
5
6      for (int sd = 0; sd < n; sd++){ // sd = sommet de départ
7          for (int i = 0; i < p.deg[sd]; i++){ // i = indice du
8              sommet suivant
9                  if (vus[sd][i]) continue;
10                 int taille_f = 1;
11                 vus[sd][i] = true;
12                 int s1 = sd;
13                 int s2 = p.adj[s1][i].sommet;
14                 proc_sommet[s1] = p.adj[s1][i];
15                 int last_j = i;
16
17                 while (s2 != sd){
18                     int j = 0;
19                     while (p.adj[s2][j].id_arete != p.adj[s1][last_j
20                     ].id_arete){
21                         j = (j+1)%p.deg[s2];
22                     }
23                 }
24             }
25         }
26     }
```

Annexe (II) (4/35) II

Annexe (II) (5/35) I

```
1         j = (j+1)%p.deg[s2];
2         int s3 = p.adj[s2][j].sommet;
3         proc_sommet[s2] = p.adj[s2][j];
4         vus[s2][j] = true;
5         s1 = s2;
6         s2 = s3;
7         taille_f++;
8         last_j = j;
9     }
10
11     f[i_face].n_sommets = taille_f;
12     f[i_face].sommets = malloc(sizeof(voisin) * taille_f
13 );
14     f[i_face].sommets[0] = proc_sommet[sd];
15     for (int fi = 1; fi < taille_f; fi++){
16         f[i_face].sommets[fi] = proc_sommet[f[i_face].
17 sommets[fi-1].sommet];
18     }
19     i_face++;
20 }
```

Annexe (II) (6/35) I

```
1
2     for (int i = 0; i < n; i++){
3         free(vus[i]);
4     }
5     free(vus);
6     free(proc_sommet);
7
8     faces_arr res = {.n = i_face, .faces = f};
9     return res;
10 }
11
12 /*
13  pour calculer le graphe de Tait :
14      on calcule l'ensemble des faces
15      a partir d'une face qcq, on fait un bfs, où les voisins de
16      la face f sont les faces qui ne partagent pas d'arête avec
17      f.
18      l'ensemble des faces parcourues est l'ensemble des sommets
19      du graphe de Tait, et les arêtes sont entre les faces qui
20      partagent un sommet.
21 */
22
23 graphe_tait calculer_graphe_tait(plongement p){
```

Annexe (II) (6/35) II

```
20 | faces_arr fa = calculer_faces(p);
```

Annexe (II) (7/35) I

```
1  int max_n = fa.n;
2  face *faces = fa.faces;
3  graphe_tait gt;
4  bool simple = true;
5
6  bool *vus = malloc(sizeof(bool) * max_n);
7  for (int i = 0; i < max_n; i++){
8      vus[i] = false;
9  }
10 for (int attempts = 0; attempts < 2; attempts++){ // il y a
    seulement 2 graphes de Tait possibles
11     int sommet_depart;
12     for (int i = 0; i < max_n; i++){
13         if (!vus[i]){
14             sommet_depart = i;
15             break;
16         }
17     }
18     int *file = malloc(sizeof(int) * max_n);
19     file[0] = sommet_depart; // a la fin du bfs, les indices
        des faces présentes dans file[0:r] sont les indices des
        faces qui sont les sommets du graphe de Tait
20     vus[sommet_depart] = true;
```

Annexe (II) (7/35) II

Annexe (II) (8/35) I

```
1      int r = 1;
2      for (int l = 0; l < r; l++){
3          // on calcule les voisins, et les ajoute à la file
4          // si on ne les a pas encore vus
5          for (int i = 0; i < max_n; i++){
6              if (vus[i]) continue;
7              if (test_faces_voisines(faces[file[l]], faces[i
8          ])){
9              vus[i] = true;
10             file[r] = i;
11             r++;
12         }
13     }
14
15     simple = true;
16     gt.n = r;
17     gt.deg = malloc(sizeof(int) * gt.n);
18     gt.adj = malloc(sizeof(int*) * gt.n);
19     gt.sommets = malloc(sizeof(face) * gt.n);
```

Annexe (II) (9/35) I

```
1      for (int i = 0; i < r; i++){
2          gt.sommets[i] = copie_face(faces[file[i]]);
3          gt.deg[i] = gt.sommets[i].n_sommets;
4          gt.adj[i] = malloc(sizeof(int) * gt.deg[i]);
5      }
6
7      int *temp_voisins = malloc(sizeof(int) * gt.n);
8      for (int i = 0; i < r; i++){
9          int nb_temp_voisins = 0;
10         for (int j = 0; j < gt.n; j++){
11             if (test_faces_voisines(gt.sommets[i], gt.
12 sommets[j])){
13                 temp_voisins[nb_temp_voisins] = j; // une
14 face ne peut pas etre voisine d'elle même
15                 nb_temp_voisins++;
16             }
17         }
18
19         if (nb_temp_voisins < gt.deg[i]) { // deux voisins
20             sont égaux, donc gt est un multigraphe
21             simple = false;
22             break;
23         }
24     }
```

Annexe (II) (9/35) II

Annexe (II) (10/35) I

```
1      for (int j = 0; j < gt.deg[i]; j++){
2          for (int k = 0; k < nb_temp_voisins; k++){
3              int s = gt.sommets[i].sommets[j].sommet;
4              for (int l = 0; l < gt.sommets[temp_voisins[
5                  k]].n_sommets; l++){
6                  if (gt.sommets[temp_voisins[k]].sommets[
7                      l].sommet == s){
8                      // seulement un seul des voisins de
9                      f lui partage le sommet s. De cette manière, les voisins
10                     sont dans le meme ordre que les sommets autour de f, ce qui
11                     fait que gt est un plongement combinatoire
12                     gt.adj[i][j] = temp_voisins[k];
13                 }
14             }
15         }
16     }
17     free(file);
18     free(temp_voisins);
19     if (simple) {
20         break;
21     }
```

Annexe (II) (10/35) II

```
19     free_graphe_tait(gt);  
20 }
```

Annexe (II) (11/35) I

```
1      assert(simple); // Pas de graphe de Tait simple.
2
3
4      for (int i = 0; i < fa.n; i++){
5          free(fa.faces[i].sommets);
6      }
7      free(faces);
8      free(vus);
9
10     return gt;
11 }
12
13 graphe trianguler_faces(plongement p){
14     faces_arr faces = calculer_faces(p);
15     graphe g;
16     g.n = p.n+faces.n;
17     g.adj = malloc(sizeof(int_vector*) * g.n);
18     for (int i = 0; i < p.n; i++){
19         g.adj[i] = init_ivec(p.deg[i]*2);
20         for (int j = 0; j < p.deg[i]; j++){ // chaque demie
arête est dans une et une seule face
```

Annexe (II) (12/35) I

```
1      bool done = false;
2      for (int f = 0; f < faces.n; f++){
3          for (int s = 0; s < faces.faces[f].n_sommets; s
4      ++){
5              if (faces.faces[f].sommets[s].id_arete == p.
6      adj[i][j].id_arete &&
7              faces.faces[f].sommets[s].somet == p.
8      adj[i][j].somet){
9                  append_ivec(g.adj[i], f+p.n);
10                 done = true;
11                 break;
12             }
13         }
14         if (done) break;
15     }
16     append_ivec(g.adj[i], p.adj[i][j].somet);
17 }
18 for (int i = 0; i < g.n-p.n; i++){
19     g.adj[i+p.n] = init_ivec(faces.faces[i].n_sommets);
20     for (int j = faces.faces[i].n_sommets - 1; j >= 0; j--){
21         append_ivec(g.adj[i+p.n], faces.faces[i].sommets[j].
22     sommet);
```

Annexe (II) (12/35) II

20

}

Annexe (II) (13/35) I

```
1     }
2     for (int i = 0; i < faces.n; i++) {
3         free(faces.faces[i].sommets);
4     }
5     free(faces.faces);
6     return g;
7 }
8
9 plongement gt_vers_plongement(graphe_tait gt){
10     plongement p;
11     int n = gt.n;
12
13     p.n = n;
14     p.deg = malloc(sizeof(int) * n);
15     p.adj = malloc(sizeof(voisin*) * n);
16
17     int compteur_aretes = 0;
18
19     for (int i = 0; i < n; i++){
20         p.deg[i] = gt.deg[i];
```

Annexe (II) (14/35) I

```
1      p.adj[i] = malloc(sizeof(voisin) * p.deg[i]);
2
3      for (int j = 0; j < p.deg[i]; j++){
4          int v = gt.adj[i][j];
5          int s = gt.sommets[i].sommets[j].sommet; // à chaque
           sommet de G_noeud est associée une et une seule arête de
           gt
6
7          voisin w;
8          w.sommet = v;
9          w.id_arete = s;
10         p.adj[i][j] = w;
11     }
12 }
13
14 return p;
15 }
16
17 plongement graphe_vers_plongement(graphe g){
18     int n = g.n;
19     plongement p;
20     int **indice_arete;
```

Annexe (II) (15/35) I

```
1  indice_arete = malloc(sizeof(int*) * n);
2  for (int i = 0; i < n; i++){
3      indice_arete[i] = malloc(sizeof(int) * n);
4      for (int j = 0; j < n; j++){
5          indice_arete[i][j] = -1;
6      }
7  }
8  int compteur_indice_aretes = 0;
9  p.n = n;
10 p.deg = malloc(sizeof(int) * n);
11 p.adj = malloc(sizeof(voisin*) * n);
12 for (int i = 0; i < n; i++){
13     p.deg[i] = g.adj[i]->taille;
14     p.adj[i] = malloc(sizeof(voisin) * p.deg[i]);
15     for (int j = 0; j < p.deg[i]; j++){
16         voisin v;
17         v.sommet = get_ivec(g.adj[i], j);
18         if (indice_arete[i][v.sommet] == -1){
19             indice_arete[i][v.sommet] =
20             compteur_indice_aretes;
21             indice_arete[v.sommet][i] =
22             compteur_indice_aretes;
```

Annexe (II) (16/35) I

```
1         compteur_indice_aretes++;
2     }
3     v.id_arete = indice_arete[i][v.sommet];
4     p.adj[i][j] = v;
5 }
6 }
7 for (int i = 0; i < n; i++){
8     free(indice_arete[i]);
9 }
10 free(indice_arete);
11
12 return p;
13 }
14
15 /*
16  il faut ensuite trianguler le graphe de Tait T en un graphe G,
    puis calculer son graphe des angles réduit H' de la manière
    suivante :
17  on calcule un plongement combinatoire de T (ce que l'on peut
    faire sans algorithme compliqué, étant donné sa
    construction, car on a conservé l'ordre des sommets autour
    de chaque face, ce qui donne l'ordre des voisins de chaque
    face)
```

Annexe (II) (16/35) II

- 18 *on rajoute un sommet sur chaque face de T , que l'on relie à*
- 19 *chaque sommet de T , en préservant le plongement (ce qui*
- 20 *donne G)*
- 19 *on calcule le dual de G*
- 20 *on choisit une face quelconque pour être la face extérieure*

Annexe (II) (17/35) I

```
1  */
2
3  graphe_angles_reduit calculer_graphe_angles_reduit(plongement p)
4  {
5      faces_arr faces = calculer_faces(p);
6      graphe g;
7      g.n = p.n+faces.n-1;
8      g.adj = malloc(sizeof(int_vector*) * g.n);
9      for (int i = 0; i < p.n; i++){
10         g.adj[i] = init_ivec(p.deg[i]);
11         for (int j = 0; j < p.deg[i]; j++){ // chaque demie
12            arête est dans une et une seule face
13            bool done = false;
14            for (int f = 0; f < faces.n; f++){
15                for (int s = 0; s < faces.faces[f].n_sommets; s
16                ++){
17                    if (faces.faces[f].sommets[s].id_arete == p.
18                    adj[i][j].id_arete &&
19                    faces.faces[f].sommets[s].sometet == p.
20                    adj[i][j].sometet){
21                        if (f+p.n < g.n) { // la face exterieure
22                            est la dernière de faces, il ne faut pas l'ajouter
23                            append_ivec(g.adj[i], f+p.n);
```

Annexe (II) (17/35) II

```
18         } else {  
19             append_ivec(g.adj[i], -1);  
20         }
```

Annexe (II) (18/35) I

```
1         done = true;
2         break;
3     }
4 }
5     if (done) break;
6 }
7     // on n'ajoute pas les arêtes originales de p
8 }
9 }
10 for (int i = 0; i < g.n-p.n; i++){
11     g.adj[i+p.n] = init_ivec(faces.faces[i].n_sommets);
12     for (int j = faces.faces[i].n_sommets - 1; j >= 0; j--){
13         append_ivec(g.adj[i+p.n], faces.faces[i].sommets[j].
14         sommet);
15     }
16 }
17 graphe_angles_reduit gar = {g, malloc(sizeof(int) * faces.
18     faces[faces.n - 1].n_sommets)};
19 for (int i = 0; i < faces.faces[faces.n - 1].n_sommets; i++)
20 {
21     gar.cycle_exterieur[i] = faces.faces[faces.n - 1].
22     sommets[i].sommet;
23 }
```

Annexe (II) (18/35) II

20

```
for (int i = 0; i < faces.n; i++) {
```

Annexe (II) (19/35) I

```
1      free(faces.faces[i].sommets);
2  }
3  free(faces.faces);
4  return gar;
5 }
6
7 double phi_prime(int s, double *x, graphe H){
8     double sum = 0.0;
9     for (int i = 0; i < H.adj[s]->taille; i++){
10         int j = get_ivec(H.adj[s], i);
11         if (j == -1){ continue; } // on ne prend pas la face
12         exterieure en compte
13         sum += atan(exp(x[s]-x[j])) - atan(exp(x[j]-x[s])) - PI
14         / 2.0;
15     }
16     sum += PI * 2.0;
17     return sum;
18 }
19
20 double* calculer_rayons(graphe_angles_reduit gar, double
    coeff_apprentissage, double epsilon){
21     graphe H = gar.H;
22     int n = H.n;
```

Annexe (II) (19/35) II

Annexe (II) (20/35) I

```
1  int s1 = gar.cycle_exterieur[0], s2 = gar.cycle_exterieur
   [1], s3 = gar.cycle_exterieur[2];
2  double *x = malloc(sizeof(double) * n);
3  double *dphidx = malloc(sizeof(double) * n);
4  for (int i = 0; i < n; i++){
5      x[i] = log(tan(PI/3))/2;
6  }
7  x[s1] = log(tan(PI/3));
8  x[s2] = log(tan(PI/3));
9  x[s3] = log(tan(PI/3));
10 for (int i = 0; i < n; i++){
11     if (i != s1 && i != s2 && i != s3) {
12         dphidx[i] = phi_prime(i, x, H);
13     } else {
14         dphidx[i] = 0;
15     }
16 }
17 double MSE = 0.0, last_MSE;
18 for (int i = 0; i < n; i++){
19     MSE += dphidx[i]*dphidx[i];
20 }
```

Annexe (II) (21/35) I

```
1
2 while (MSE > epsilon){
3     for (int i = 0; i < n; i++){
4         if (i != s1 && i != s2 && i != s3) {
5             x[i] -= coeff_apprentissage * dphidx[i];
6         }
7     }
8     for (int i = 0; i < n; i++){
9         if (i != s1 && i != s2 && i != s3) {
10             dphidx[i] = phi_prime(i, x, H);
11         } else {
12             dphidx[i] = 0;
13         }
14     }
15     last_MSE = MSE;
16     MSE = 0.0;
17     for (int i = 0; i < n; i++){
18         MSE += dphidx[i]*dphidx[i];
19     }
20 }
```

Annexe (II) (22/35) I

```
1    double *r = malloc(sizeof(double) * n);
2    for (int i = 0; i < n; i++){
3        r[i] = exp(x[i]);
4    }
5    free(x);
6    free(dphidx);
7    return r;
8 }
9
10 vec* calculer_positions(double *r, graphe_angles_reduit gar){
11     int n = gar.H.n;
12     vec* positions = malloc(sizeof(vec) * gar.H.n);
13     bool* places = malloc(sizeof(bool) * gar.H.n);
14     for (int i = 0; i < gar.H.n; i++){
15         places[i] = false;
16     }
17     int t1 = -1;
18     int s1 = gar.cycle_exterieur[0], s2 = gar.cycle_exterieur
19         [1];
20     for (int i = 0; i < gar.H.n; i++){
21         for (int j = 0; j < gar.H.adj[i]->taille; j++){
```

Annexe (II) (23/35) I

```
1         if (get_ivec(gar.H.adj[i], j) == s1 &&
2             get_ivec(gar.H.adj[i], (j + 1) % gar.H.adj[i]->
3             taille) == s2){
4             t1 = i;
5             break;
6         }
7     }
8     positions[s1].x = 0;
9     positions[s1].y = 0;
10    positions[s2].x = r[s1] + r[s2];
11    positions[s2].y = 0;
12    positions[t1].x = r[s1];
13    positions[t1].y = r[t1];
14    places[s1] = true; places[s2] = true; places[t1] = true;
15    int *file = malloc(sizeof(int) * gar.H.n);
16    file[0] = s1;
17    file[1] = s2;
18    file[2] = t1;
19    int pointeur_gauche = 0, pointeur_droite = 3;
20    while (pointeur_gauche < pointeur_droite){
```

Annexe (II) (24/35) I

```
1      int s = file[pointeur_gauche];
2      int j = 0;
3      int deg = gar.H.adj[s]->taille;
4      while (get_ivec(gar.H.adj[s], j) == -1 || places[
get_ivec(gar.H.adj[s], j)] == false){ // on saute la face
ext
5          j = (j+1)%deg;
6      }
7      bool sens_inverse = false;
8      for (int d = 0; d < deg - 1; d++){
9          int voisin = get_ivec(gar.H.adj[s], j);
10         j = (j+1)%deg;
11         int w = get_ivec(gar.H.adj[s], j);
12         if (w == -1){
13             sens_inverse = true;
14             j = (deg + j - 1)%deg;
15             break;
16         }
17         if (places[w]) {
18             continue;
19         }
```

Annexe (II) (25/35) I

```
1      // 's' and 'w' sont voisins dans H, donc leurs
    cercles sont orthogonaux
2      double r1 = sqrt(r[s]*r[s] + r[w]*r[w]);
3
4      // 'voisin' et 'w' sont voisins de 's', donc leurs
    cercles sont tangents
5      double r2 = r[voisin] + r[w];
6
7      double x1 = positions[s].x;
8      double y1 = positions[s].y;
9      double x2 = positions[voisin].x;
10     double y2 = positions[voisin].y;
11
12     double dx = x2 - x1;
13     double dy = y2 - y1;
14
15     double rDiffCar = r1*r1 - r2*r2;
16     double xDiffCar = x2*x2 - x1*x1;
17     double yDiffCar = y2*y2 - y1*y1;
18     double denom = 2.0 * (dx*dx + dy*dy);
19     double crois = y1*x2 - y2*x1;
```

Annexe (II) (26/35) I

```
1      vec barycentre;
2      barycentre.x = (dx * (rDiffCar + xDiffCar + yDiffCar
3      ) - 2.0 * dy * crois) / denom;
4      barycentre.y = (dy * (rDiffCar + xDiffCar + yDiffCar
5      ) + 2.0 * dx * crois) / denom;
6
7      vec perp;
8      perp.x = positions[s].y - positions[voisin].y;
9      perp.y = positions[voisin].x - positions[s].x;
10     perp = normaliser(perp);
11
12     double di = dist(positions[s], barycentre);
13     double scale = r1*r1 - di*di;
14     double n_len = sqrt(scale);
15     perp.x *= n_len;
16     perp.y *= n_len;
17
18     positions[w] = add(barycentre, perp);
19     places[w] = true;
20     file[pointeur_droite] = w;
21     pointeur_droite++;
22 }
```

Annexe (II) (27/35) I

```
1      if (sens_inverse){ // on est tombés sur la face  
2      exterieure, on ne peut pas la placer, donc on repart dans l  
3      'autre sens pour placer le reste des voisins.  
4      for (int d = 0; d < deg - 2; d++){  
5          int voisin = get_ivec(gar.H.adj[s], j);  
6          j = (deg + j - 1)%deg;  
7          if (places[get_ivec(gar.H.adj[s], j)]) {  
8              continue;  
9          }  
10         int w = get_ivec(gar.H.adj[s], j);  
11  
12         double r1 = sqrt(r[s]*r[s] + r[w]*r[w]);  
13  
14         double r2 = r[voisin] + r[w];  
15  
16         double x1 = positions[s].x;  
17         double y1 = positions[s].y;  
18         double x2 = positions[voisin].x;  
19         double y2 = positions[voisin].y;  
20  
21         double dx = x2 - x1;  
22         double dy = y2 - y1;
```

Annexe (II) (28/35) I

```
1      double rDiffCar = r1*r1 - r2*r2;
2      double xDiffCar = x2*x2 - x1*x1;
3      double yDiffCar = y2*y2 - y1*y1;
4      double denom = 2.0 * (dx*dx + dy*dy);
5      double crois = y1*x2 - y2*x1;
6
7
8      vec barycentre;
9      barycentre.x = (dx * (rDiffCar + xDiffCar + yDiffCar
10 ) - 2.0 * dy * crois) / denom;
11      barycentre.y = (dy * (rDiffCar + xDiffCar + yDiffCar
12 ) + 2.0 * dx * crois) / denom;
13
14      vec perp;
15      perp.x = positions[s].y - positions[voisin].y;
16      perp.y = positions[voisin].x - positions[s].x;
17      perp = normaliser(perp);
18
19      double di = dist(positions[s], barycentre);
20      double scale = r1*r1 - di*di;
21      // if (scale < 0) scale = 0;
22      double n_len = sqrt(scale);
```

Annexe (II) (29/35) I

```
1      perp.x *= n_len;
2      perp.y *= n_len;
3
4      positions[w] = diff(barycentre, perp);
5      places[w] = true;
6      file[pointeur_droite] = w;
7      pointeur_droite++;
8  }}
9      pointeur_gauche++;
10 }
11
12 free(file);
13 free(places);
14 return positions;
15 }
16
17 void dessiner_inter(vec pos, double dir, double stroke_width,
18 FILE *svg){
19     double epsilon = stroke_width;
20     vec e_eps = {epsilon * cos(dir), epsilon * sin(dir)};
```

Annexe (II) (29/35) II

20

```
fprintf(svg, "<path d=\"M %lf %lf L %lf %lf\" stroke-width  
=\"%lf\" stroke=\"white\"/>\n", pos.x - e_eps.x, pos.y -  
e_eps.y, pos.x + e_eps.x, pos.y + e_eps.y, stroke_width *  
3);
```

Annexe (II) (30/35) I

```
1      fprintf(svg, "<path d=\"M %lf %lf L %lf %lf\" stroke-width
2      =\"%lf\" stroke=\"blue\"/>\n", pos.x - e_eps.x, pos.y -
3      e_eps.y, pos.x + e_eps.x, pos.y + e_eps.y, stroke_width);
4  }
5
6  void calculer_svg(vec *pos, double *r, graphe_tait gt,
7  plongement p, bool start_over, char* filename){
8      FILE *f = fopen(filename, "w");
9      double stroke_width = 0.008;
10     fprintf(f, "<svg xmlns=\"http://www.w3.org/2000/svg\" width
11     =\"400\" height=\"400\" viewBox=\"0 0 3 3\" stroke=\"blue\"
12     stroke-width=\"%lf\" fill=\"none\">\n", stroke_width);
13     int n = 0;
14     for (int i = 0; i < gt.n; i++){
15         for (int j = 0; j < gt.deg[i]; j++){
16             if (i < gt.adj[i][j]) n++;
17         }
18     }
19     vec *pos_intersections = malloc(sizeof(vec) * n);
20     double **ddir = malloc(sizeof(double*) * gt.n);
21     bool *initied = malloc(sizeof(bool) * n);
22     for (int i = 0; i < n; i++){
23         initied[i] = false;
```

Annexe (II) (30/35) II

```
19     }  
20     for (int i = 0; i < gt.n; i++){
```

Annexe (II) (31/35) I

```
1      ddir[i] = malloc(sizeof(double) * gt.deg[i]);
2  }
3
4  for (int i = 0; i < gt.n; i++){
5      for (int j = 0; j < gt.deg[i]; j++){
6          int inter_idx = gt.sommets[i].sommets[j].sommet;
7          if (!initied[inter_idx]) {
8              initied[inter_idx] = true;
9              pos_intersections[inter_idx].x = (pos[i].x * r[
gt.adj[i][j]] + pos[gt.adj[i][j]].x * r[i]) / (r[i] + r[gt.
adj[i][j]]);
10             pos_intersections[inter_idx].y = (pos[i].y * r[
gt.adj[i][j]] + pos[gt.adj[i][j]].y * r[i]) / (r[i] + r[gt.
adj[i][j]]);
11         }
12     }
13 }
14
15 // il devrait etre possible de faire un dfs, qui dessine
16 tout le noeud
17 for (int i = 0; i < gt.n; i++){
18     for (int j = 0; j < gt.deg[i]; j++){
19         int i0 = gt.sommets[i].sommets[j].sommet;
```

Annexe (II) (31/35) II

```
19         int i1 = gt.sommets[i].sommets[(j+1)%gt.sommets[i].  
n_sommets].sommet;  
20         double alpha = angle(pos_intersections[i0], pos[i],  
pos_intersections[i1]);
```

Annexe (II) (32/35) I

```
1      ddir[i][(j+1)%gt.sommets[i].n_sommets] = alpha; //  
on utilise que le noeud est alterné ; alpha vaut déjà  
déviaton + 90° (alternance)  
2      if (sin(PI/2 - alpha/2) == 0){  
3          fprintf(f, "<path d=\"M %lf %lf L %lf %lf\"/>\n"  
        , pos_intersections[i0].x, pos_intersections[i0].y,  
        pos_intersections[i1].x, pos_intersections[i1].y);  
4      } else {  
5          double rayon_arc = r[i] * fabs(sin(alpha/2) /  
        sin(PI/4 - alpha/2));  
6          int large = 0;  
7          if (alpha >= 3 * PI / 2) large = 1;  
8          int sweep = 1;  
9          if (alpha <= PI / 2) sweep = 0;  
10         fprintf(f, "<path d=\"M %lf %lf A %lf %lf 0 %d %  
        d %lf %lf\"/>\n", pos_intersections[i0].x,  
        pos_intersections[i0].y, rayon_arc, rayon_arc, large, sweep  
        , pos_intersections[i1].x, pos_intersections[i1].y);  
11     }  
12 }  
13 }  
14  
15 bool* vus = malloc(sizeof(bool) * p.n);
```

Annexe (II) (32/35) II

```
16   for (int i = 0; i < p.n; i++){  
17       vus[i] = false;  
18   }  
19   int n_vus = 0;  
20   int s0, s1 = 0, fa = 0;
```

Annexe (II) (33/35) I

```
1  vec centre_plus_e_x = {pos[0].x+1, pos[0].y};
2
3  double dir = PI/2 - angle(pos_intersections[0], pos[0],
4  centre_plus_e_x) - PI/4 + (start_over?PI/2:0);
5  dessiner_inter(pos_intersections[0], dir, stroke_width, f);
6  vus[0] = true;
7  n_vus++;
8  int k = 0;
9  bool sens_inverse = false;
10 while (n_vus < p.n && k < 1000) {
11     s0 = s1;
12     for (int i = 0; i < gt.deg[fa]; i++){
13         if (gt.sommets[fa].sommets[i].somet == s0){
14             int j; double dev;
15             if (sens_inverse) {
16                 j = (i+gt.deg[fa]-1)%gt.deg[fa];
17                 dev = -ddir[fa][i];
18             } else {
19                 j = (i+1)%gt.deg[fa];
20                 dev = ddir[fa][j];
21             }
22         }
23     }
24 }
```

Annexe (II) (34/35) I

```
1         s1 = gt.sommets[fa].sommets[j].sommet;
2         dir += dev;
3         if (!vus[s1]){
4             dessiner_inter(pos_intersections[s1], dir,
5 stroke_width, f);
6             vus[s1] = true;
7             n_vus++;
8         }
9         fa = gt.adj[fa][j];
10        sens_inverse = !sens_inverse;
11        break;
12    }
13    k++;
14}

15
16fprintf(f, "</svg>\n");
17fclose(f);
18for (int i = 0; i < gt.n; i++){
19    free(ddir[i]);
20}
```

Annexe (II) (35/35) I

```
1    free(ddir);  
2    free(vus);  
3    free(pos_intersections);  
4    free(inited);  
5 }
```